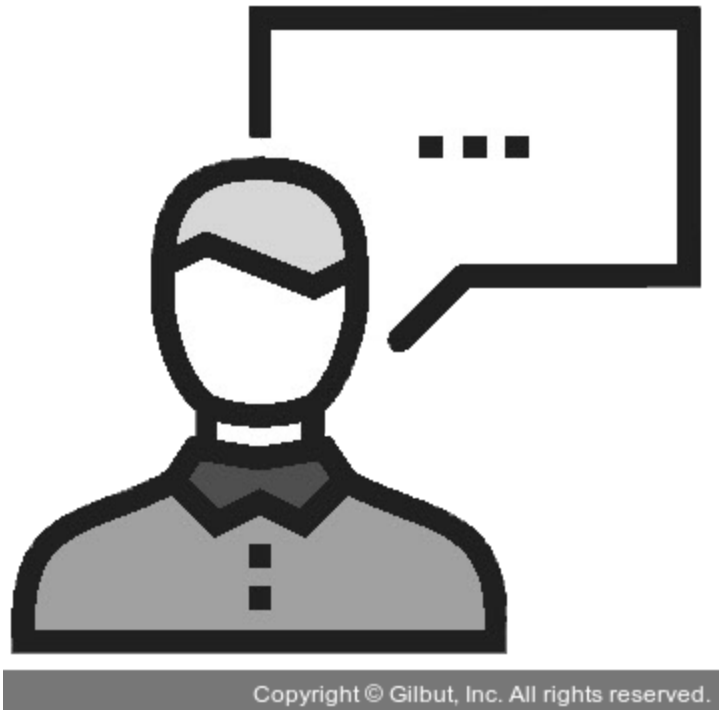
UNIT 14

numpy를 활용한 나만의 프로젝트 만들기



⦿ 실습 데이터 Unit14/unit14-01.ipynb, age.csv

지난 시간에 우리는 `numpy` 라이브러리를 사용하는 방법을 배웠습니다. 이번 시간에는 `numpy`를 활용하여 '우리 동네 인구 구조와 가장 비슷한 곳을 찾는' 간단한 프로젝트를 만들겠습니다. 특히 이번 프로젝트는 여러분이 이 책을 다 읽고 난 후 여러분만의 데이터 프로젝트를 만드는 데 도움이 되도록, 프로젝트를 점진적으로 완성하는 과정을 보여주려고 합니다. 이번 프로젝트를 진행하면서 나만의 프로젝트에 대한 구상을 함께 해보길 바랍니다.

1 관심 있는 데이터 찾기

나만의 데이터 프로젝트를 시작하는 데는 다양한 절차가 있지만, 처음 시작하는 단계에서는 관심 있는 데이터를 찾는 것에서부터 시작하는 것이 좋습니다.

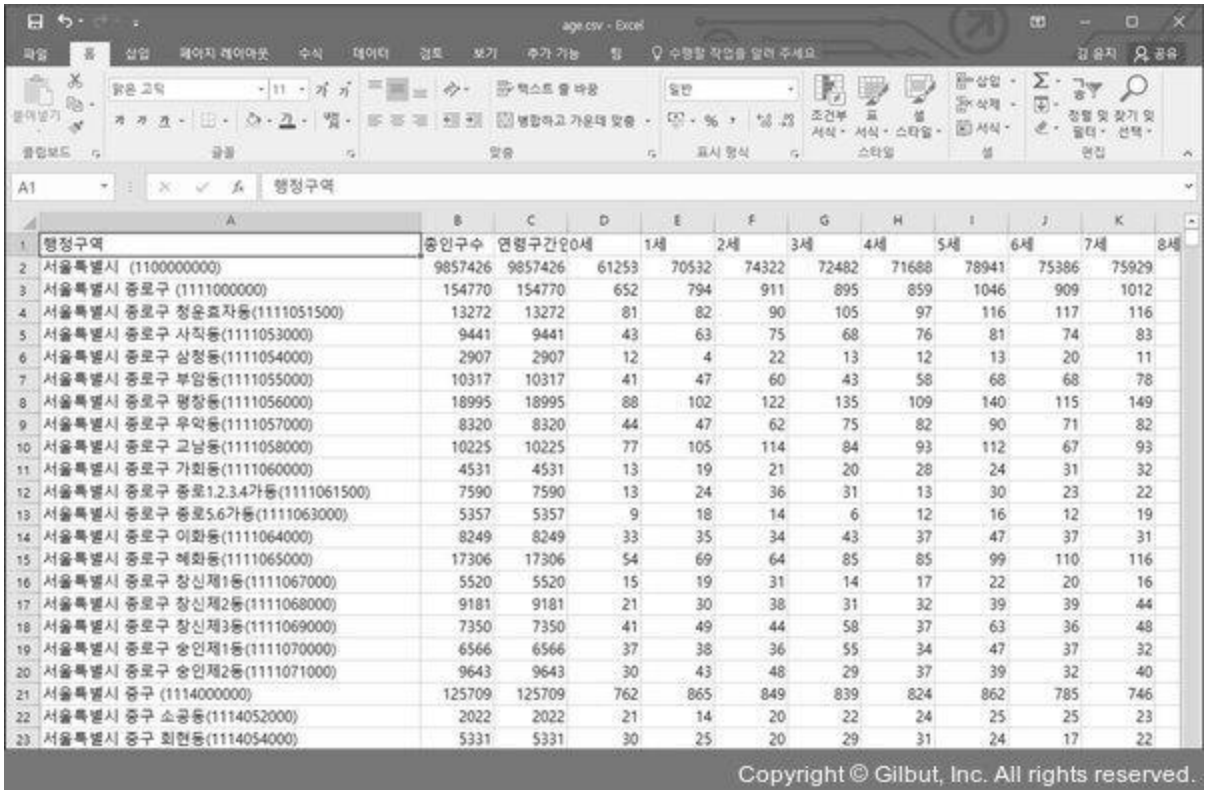
최근 공공기관에서 많은 공공데이터를 제공하기 때문에 이전에 소개했던 공공데이터포털(www.data.go.kr/)을 이용하거나, 인터넷 검색을 해보면 관심 있는 분야의 데이터를 쉽게 찾을 수 있습니다.



그림 14-1 공공데이터포털

2 데이터 살펴보며 질문하기

관심 있는 데이터를 살펴보면 자연스럽게 질문이 떠오르기 마련입니다. Unit 7에서 다뤘던 인구 데이터를 다시 한번 살펴보겠습니다.



age.csv - Excel

A1 행정구역

	A	B	C	D	E	F	G	H	I	J	K
1	행정구역	총인구수	연령구간인구수	0세	1세	2세	3세	4세	5세	6세	7세	8세
2	서울특별시 (1100000000)	9857426	9857426	61253	70532	74322	72482	71688	78941	75386	75929	
3	서울특별시 종로구 (1111000000)	154770	154770	652	794	911	895	859	1046	909	1012	
4	서울특별시 종로구 청운효자동(1111051500)	13272	13272	81	82	90	105	97	116	117	116	
5	서울특별시 종로구 사직동(1111053000)	9441	9441	43	63	75	68	76	81	74	83	
6	서울특별시 종로구 삼청동(1111054000)	2907	2907	12	4	22	13	12	13	20	11	
7	서울특별시 종로구 부암동(1111055000)	10317	10317	41	47	60	43	58	68	68	78	
8	서울특별시 종로구 평창동(1111056000)	18995	18995	88	102	122	135	109	140	115	149	
9	서울특별시 종로구 무악동(1111057000)	8320	8320	44	47	62	75	82	90	71	82	
10	서울특별시 종로구 교남동(1111058000)	10225	10225	77	105	114	84	93	112	67	93	
11	서울특별시 종로구 가회동(1111060000)	4531	4531	13	19	21	20	28	24	31	32	
12	서울특별시 종로구 종로1.2.3.4가동(1111061500)	7590	7590	13	24	36	31	13	30	23	22	
13	서울특별시 종로구 종로5.6가동(1111063000)	5357	5357	9	18	14	6	12	16	12	19	
14	서울특별시 종로구 이화동(1111064000)	8249	8249	33	35	34	43	37	47	37	31	
15	서울특별시 종로구 혜화동(1111065000)	17306	17306	54	69	64	85	85	99	110	116	
16	서울특별시 종로구 창신제1동(1111067000)	5520	5520	15	19	31	14	17	22	20	16	
17	서울특별시 종로구 창신제2동(1111068000)	9181	9181	21	30	38	31	32	39	39	44	
18	서울특별시 종로구 창신제3동(1111069000)	7350	7350	41	49	44	58	37	63	36	48	
19	서울특별시 종로구 숭인제1동(1111070000)	6566	6566	37	38	36	55	34	47	37	32	
20	서울특별시 종로구 숭인제2동(1111071000)	9643	9643	30	43	48	29	37	39	32	40	
21	서울특별시 중구 (1114000000)	125709	125709	762	865	849	839	824	862	785	746	
22	서울특별시 중구 소공동(1114052000)	2022	2022	21	14	20	22	24	25	25	23	
23	서울특별시 중구 회현동(1114054000)	5331	5331	30	25	20	29	31	24	17	22	

Copyright © Gilbut, Inc. All rights reserved.

그림 14-4 Unit 7에서 살펴본 age.csv 파일

3 질문을 명확한 문제로 정의하기

그렇지만 앞에서 던진 질문들은 조금 더 다듬을 필요가 있습니다. 예를 들어 '전국에서 가장 영유아들이 많이 사는 지역은 어디일까?'라는 질문은 '전국에 있는 읍면동 중 만 0세 이상 6 세 이하의 인구 비율이 높은 상위 10 곳은 어디일까?'라는 질문으로 바꿀 수 있습니다. 또 청소년은 만 13세 이상 만 19세 미만의 인구, 청년은 만 19세 이상의 인구로 구체적인 나이를 언급한 질문으로 바꿀 수 있겠지요. 이렇게 문제를 명확히 정의하고 나면 현재 갖고 있는 데이터로 문제를 해결할 수 있는지를 판단할 수 있을 뿐 아니라, 문제를 해결하기 위한 알고리즘을 설계하는 것도 훨씬 수월해집니다.

앞서 모든 질문에 대한 답을 찾아볼 수 있겠지만, 여기에는 한 가지 질문만을 선택해서 명확한 문제로 정의하겠습니다. 질문을 선택할 때는 가장 궁금하면서도 현재 지식과 능력으로 해결 가능한 것을 선택합니다.

개인적으로 내가 살고 있는 동네의 인구 구조와 가장 비슷한 동네를 찾는 질문이 가장 재미있을 것 같습니다. 이 질문에 대한 답을 찾으면 우리 동네뿐만 아니라 전국에 있는 모든 지역을 대상으로 가장 비슷한 지역이 어디인지 알 수 있을 겁니다. 그리고 그 결과는 상권을 분석하거나, 3~5년 후 발생할 수 있는 학교 수요·공급 문제를 예측하는 데 도움을 줄 수 있을 것이기 때문입니다.

그러면 '우리 동네의 인구 구조와 가장 비슷한 지역은 어디일까?'라는 질문을 '전국에서 신도림동의 연령별 인구 구조와 가장 형태가 비슷한 지역은 어디일까?'라고 다듬었습니다. 물론 문제를 좀 더 명확하게 정의할 수도 있지만, 이 정도로 생각해도 알고리즘을 생각하는 데 큰 무리는 없을 것 같습니다.

4 알고리즘 설계하기

이제 문제를 해결하기 위한 절차, 즉 알고리즘을 설계하겠습니다. 알고리즘 설계에 앞서서 우리가 해결하려는 문제가 현재 가진 데이터(입력)로 해결할 수 있는지 확인해야 합니다. 만약 현재 데이터로 불충분하다면 문제 해결에 필요한 데이터를 더 수집해야 하니까요.

우리가 해결하려는 문제는 '우리 동네의 인구 구조와 가장 비슷한 지역은 어디일까?'입니다. 문제 해결을 위해 필요한 데이터는 지역별 인구수를 연령대, 성별을 기준으로 구분한 자료입니다. 우리는 이러한 자료를 행정안전부 홈페이지에서 얻을 수 있었습니다(Unit 7). 따라서 문제 해결을 위해 필요한 데이터를 충분히 갖고 있다고 볼 수 있습니다.

현재 수집된 데이터로 문제를 해결할 수 있다면 문제를 해결하기 위한 절차를 차례대로 생각해 봅니다. 이 단계는 코드로 작성하는 것을 생각하는 것이 아니고 문제를 해결하기 위한 논리적인 절차를 생각하는 것입니다.

5 알고리즘을 코드로 표현하기

이제 알고리즘의 각 단계를 코드로 표현하겠습니다.

■ 1 | 데이터를 읽어온다.

|

■ 1 | 데이터를 읽어온다.

데이터는 Unit 7에서 수집했던 데이터를 그대로 사용하겠습니다. 따라서 데이터를 읽어오는 코드도 동일합니다.

```
import csv
f = open('age.csv')
data = csv.reader(f)
for row in data :
    print(row)
```

실행 결과

```
['행정구역', '총인구수', '연령구간인구수', '0세', '1세', '2세', '3세', '4세', '5세', '6세', '7세', '8세', (생략), '94세', '95세', '96세', '97세', '98세', '99세', '100세 이상']
['서울특별시  (1100000000)', '9765623', '9765623', '54719', '60805', '67262', '71433', '70251', '69728', '76957', '73813', '74748', '70652', '75180', '79733', '74074', '71698', '79681', '82773', '83012', '92791', '108139', '108040', '114742', '122499', '129725', '141389', '150560', '156527', '165963', '165471', '153068', '149846', '149513', '
```

그리고 데이터 처리에 불필요한 헤더 부분을 제외시키고자 `next()` 함수를 사용합니다.

```
import csv
f = open('age.csv')
data = csv.reader(f)
next(data)
for row in data :
    print(row)
```

■ 2 | 궁금한 지역의 이름을 입력받는다.

`name` 변수에 궁금한 지역의 이름을 입력받겠습니다. 입력받는 코드의 위치는 어디가 좋을지 생각해 보고 적절한 위치에 입력하세요.

```
name = input('인구 구조가 알고 싶은 지역의 이름(읍면동 단위)을 입력해주세요 : ')
```

■ 3 | 궁금한 지역의 인구 구조를 저장한다.

```
import csv
f = open('age.csv')
data = csv.reader(f)
next(data)
home = []                          # 입력받은 지역의 데이터를 저장할 리스트 생성
name = input('인구 구조가 알고 싶은 지역의 이름(읍면동 단위)을 입력해주세요 : ')
for row in data :
    if name in row[0] :            # 입력받은 지역의 이름이 포함된 행 찾기
        for i in row[3:] :         # 3번 인덱스 값부터 슬라이싱
            home.append(int(i))    # 입력받은 지역의 데이터를 home에 저장
print(home)                        # home에 저장된 데이터 출력
```

실행 결과

```
인구 구조가 알고 싶은 지역의 이름(읍면동 단위)을 입력해주세요 : 신도림──▶ 입력
[333, 358, 408, 418, 390, 416, 406, 444, 396, 408, 408, 384, 356, 370, 331, 334, 350, 358, 406, 36
6, 397, 440, 448, 487, 460, 471, 438, 499, 443, 502, 539, 538, 603, 609, 654, 753, 765, 790, 747,
794, 658, 698, 662, 664, 719, 703, 631, 648, 622, 572, 552, 505, 505, 543, 530, 474, 531, 562, 56
2, 523, 512, 468, 404, 515, 386, 390, 393, 252, 298, 265, 286, 286, 179, 164, 156, 154, 176, 143,
132, 111, 112, 90, 91, 71, 64, 42, 45, 43, 24, 27, 18, 16, 18, 10, 4, 11, 3, 3, 3, 2, 7]
```

■ 4 | 궁금한 지역의 인구 구조와 가장 비슷한 인구 구조를 가진 지역 찾기

이제 우리 프로젝트에서 가장 중요한 부분입니다. 특정 지역의 인구 구조와 가장 비슷한 인구 구조를 가진 지역을 어떻게 찾을 수 있을까요? 여러분도 잠깐 고민해보기 바랍니다.

만약 궁금한 지역을 A, 비교할 지역을 B라고 하면 A의 0세 인구수와 B의 0세 인구수의 차이, A의 1세 인구수와 B의 1세 인구수의 차이, 이런 식으로 A의 100세 이상 인구수와 B의 100세 이상 인구수의 차이를 모두 더했을 때 그 차이가 가장 작은 지역을 찾으면 어떨까요?

이것을 다시 알고리즘으로 표현하겠습니다.

❶ 전국의 모든 지역 중 한 곳(B)을 선택한다.

❷ 궁금한 지역 A의 0세 인구수에서 B의 0세 인구수를 뺀다.

❸ ❷를 100세 이상 인구수에 해당하는 값까지 반복한 후 각각의 차이를 모두 더한다.

❹ 전국의 모든 지역에 대해 반복하며 그 차이가 가장 작은 지역을 찾는다.

혹시 이 알고리즘을 보면서 문제가 될 것 같다고 생각이 드는 부분이 있나요?

다음 질문에 대한 답을 확인하며 이 알고리즘을 수정합시다.

■ 5 | 가장 비슷한 곳의 인구 구조와 궁금한 지역의 인구 구조를 시각화한다.

드디어 마지막 단계입니다. 데이터를 시각화할 때는 다양한 형태 중 적절한 형태를 고르는 것이 중요한데요, 여기에서는 두 개의 그래프를 비교할 목적이므로 가장 기본 그래프인 꺾은선 그래프로 표현하겠습니다. 드디어 우리 프로젝트의 결과가 나옵니다!

```
import matplotlib.pyplot as plt
plt.plot(home)
plt.plot(result)
plt.show()
```

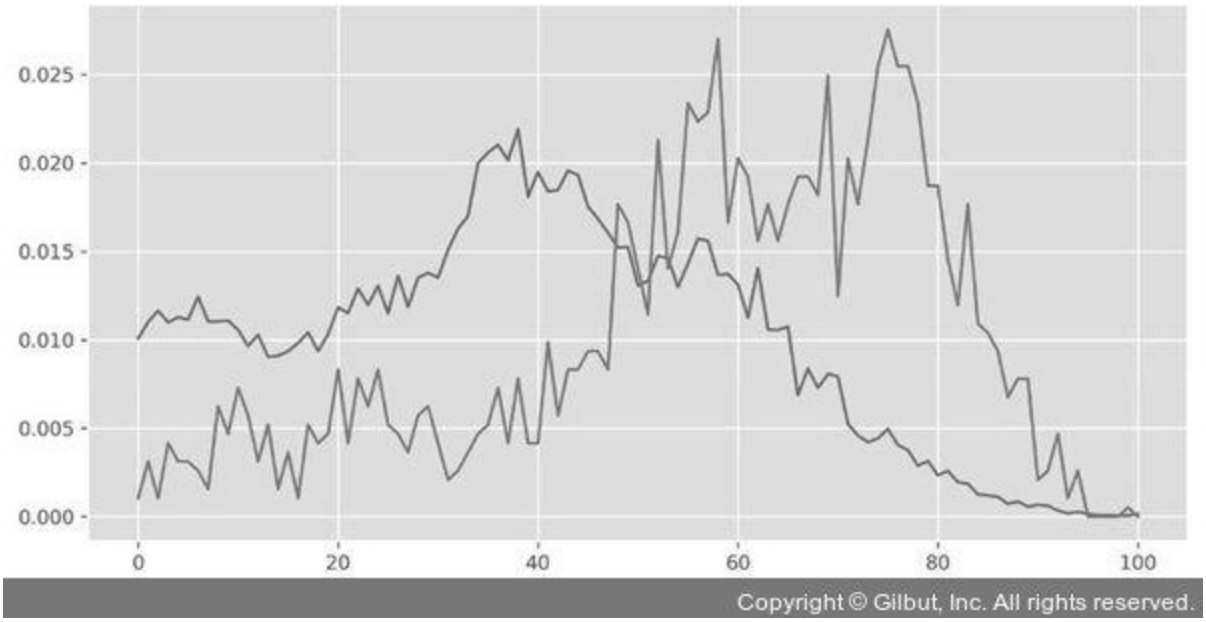


그림 14-9 두 지역의 인구 구조를 그래프로 나타낸 결과